Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas





Título del Proyecto:

CLOUDGRID DATACENTER

FASE 3: SECURE BACKBONE

PONDERACIÓN: 30

Horas Aproximadas: 60

Índice

1. Resumen Ejecutivo **3**

2. Competencia que desarrollaremos **3**

3. Objetivos del Aprendizaje **4**

3.1 Objetivo General 4

3.2 Objetivos Específicos 4

4. Enunciado del Proyecto **5**

4.1 Descripción del problema a resolver 5

4.2 Alcance del proyecto 16

5. Desarrollo de Habilidades Blandas **17**

5.1 Proyectos Individuales 17

5.2 Resumen de Puntuaciones 18

5.3 Observaciones 19

1. Resumen Ejecutivo

Tras consolidar en la Fase 1 la infraestructura física y lógica de un único centro de datos, y en la Fase 2 la expansión hacia una red global de Data Centers (Tabla Hash para el servidor DNS interno, Matriz Dispersa para el tráfico entre centros, Min-Heap para la planificación de trabajos y un Grafo No Dirigido Ponderado con BFS y DFS para el backbone), CloudGrid enfrenta un nuevo reto: operar esa red de forma óptima y verificable. Los enlaces de fibra son costosos y el backbone tiene redundancia; es necesario saber cuál es la red troncal de menor latencia que mantiene conectados a todos los Data Centers. Además, cada trabajo ejecutado (despliegues, migraciones, réplicas, respaldos y parches) modifica registros DNS y consume tráfico, por lo que debe quedar registrado de forma inmutable y auditable. Finalmente, el estado de la red, del DNS y de los trabajos pendientes debe conservarse entre ejecuciones ocupando el menor espacio posible.

La Fase 3, denominada CloudGrid Secure Backbone, extiende el sistema en C++ nativo con tres componentes: los algoritmos de Árbol de Expansión Mínima de Prim y Kruskal sobre el grafo del backbone, que obtienen la red troncal de menor latencia y el árbol de replicación global; una Blockchain con prueba de trabajo, con un Árbol de Merkle por bloque, que sella la bitácora de trabajos ejecutados y permite detectar cualquier manipulación; y un mecanismo de compresión LZW que persiste en archivos .edd el estado de la red, del DNS y de la cola de trabajos. Todos los componentes colaboran con las estructuras de la Fase 2 y la carga masiva migra a un único archivo JSON, más legible y jerárquico.

2. Competencia que desarrollaremos

A través de la tercera fase del sistema CloudGrid DataCenter, se consolidarán competencias en el diseño e implementación de algoritmos voraces sobre grafos (Prim y Kruskal), incluyendo la estructura de conjuntos disjuntos (Union-Find) que Kruskal requiere; en la construcción de cadenas de bloques enlazadas criptográficamente y de Árboles de Merkle; y en la compresión de datos sin pérdida con el algoritmo LZW. Todo ello se realiza en C++ sin el uso de contenedores de la librería estándar (STL), con control riguroso de la memoria dinámica y de los punteros. Se espera además que el estudiante reutilice con criterio las estructuras de la Fase 2 (por ejemplo, el Min-Heap como cola de prioridad y la Tabla Hash como diccionario).

Adicionalmente, se fortalecerán capacidades de seguridad aplicada (funciones hash SHA-256, prueba de trabajo y detección de manipulación de datos), de serialización de estructuras en memoria a un formato de texto estructurado sin archivos intermedios, de lectura y validación de archivos JSON, y de generación automatizada de reportes con Graphviz. Esto fomenta el análisis de complejidad, la abstracción de problemas reales de infraestructura en la nube y el desarrollo de código limpio, modular y mantenible.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Extender la plataforma CloudGrid DataCenter en C++ hacia una red global optimizada, auditable y persistente, calculando la red troncal de menor latencia con los algoritmos de Prim y Kruskal, registrando de forma inmutable los trabajos ejecutados en una Blockchain con Árbol de Merkle, y almacenando el estado del sistema mediante compresión LZW, integrando interfaz gráfica y reportes visuales con Graphviz.

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

- Implementar el algoritmo de Prim sobre la lista de adyacencia del backbone para obtener el Árbol de Expansión Mínima desde un Data Center de origen.
- Implementar el algoritmo de Kruskal con una estructura de conjuntos disjuntos propia (Union-Find) y comparar su resultado, orden de selección y complejidad con los de Prim.
- Construir una Blockchain como lista enlazada simple con inserción por el frente, aplicando prueba de trabajo con SHA-256 y persistencia de cada bloque en archivos JSON.
- Construir un Árbol de Merkle por bloque y utilizarlo para detectar manipulaciones y verificar la inclusión de una transacción.
- Implementar el algoritmo de compresión y descompresión LZW, con diccionario dinámico propio, para persistir la red, el DNS y los trabajos pendientes en archivos .edd.
- Integrar las nuevas estructuras con la Tabla Hash, la Matriz Dispersa, el Min-Heap y el Grafo de la Fase 2 mediante un flujo de operación coherente.
- Integrar Graphviz para visualizar los Árboles de Expansión Mínima, el Árbol de Merkle y la Blockchain (bloques íntegros en verde y manipulados en rojo), y documentar el diseño con Git/GitHub y un manual técnico versionado.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

Arquitectura y Mapeo de Estructuras de la Fase 3

El núcleo de CloudGrid Secure Backbone incorpora las siguientes estructuras y algoritmos nuevos, cada uno con un rol técnico específico y no intercambiable:

Estructura / Algoritmo	Entidad Modelada	Clave / Índice	Justificación Técnica y Operaciones
Árbol de Expansión Mínima: Prim y Kruskal	Red troncal de menor latencia del backbone	latencia_ms de cada enlace (peso de la arista)	Prim crece desde un DC de origen eligiendo siempre el enlace más barato que sale del árbol; Kruskal ordena los enlaces y los acepta si no forman ciclo (Union-Find). Solo se consideran enlaces activos; si el grafo es disconexo se obtiene un bosque.
Blockchain (Lista Enlazada Simple, inserción por el frente)	Bitácora inmutable de trabajos ejecutados	INDEX; HASH SHA-256 con prefijo "0000" (prueba de trabajo)	Cada bloque almacena los trabajos ejecutados pendientes de auditar, enlaza al hash del bloque anterior y se persiste como JSON en la carpeta blocks/. Verificación de integridad al iniciar el programa.
Árbol de Merkle	Resumen criptográfico de los trabajos de un bloque	SHA-256 de cada transacción (hojas)	Árbol binario construido de abajo hacia arriba. Su raíz (ROOTMERKLE) forma parte del hash del bloque y permite detectar manipulaciones y verificar la inclusión de un trabajo.
Compresión LZW (archivos .edd)	Persistencia de la red, el DNS y los trabajos pendientes	Texto serializado en memoria → secuencia de códigos	Diccionario dinámico inicializado con 256 entradas. Se comprime al cerrar el programa a partir de las estructuras en memoria y se descomprime al iniciar para reconstruirlas.

Integración con las Estructuras de la Fase 2

Las estructuras de la Fase 2 se conservan y adquieren nuevas responsabilidades dentro de la Fase 3:

Estructura de la Fase 2	Rol en la Fase 3
Grafo No Dirigido Ponderado (Lista de Adyacencia)	Es la entrada de Prim y Kruskal. Cada enlace incorpora el campo activo, de modo que la simulación de falla de enlace de la Fase 2 también afecta al árbol de expansión. Se serializa en red.edd.
Min-Heap (planificador de trabajos)	Al despachar un trabajo (extracción de la raíz) este se ejecuta y genera una transacción pendiente de auditoría. Se recomienda reutilizar el Min-Heap como cola de prioridad de Prim y para ordenar las aristas de Kruskal. Los trabajos pendientes se serializan en trabajos.edd.
Tabla Hash (DNS interno)	Los trabajos DEPLOY y MIGRAR publican o actualizan registros DNS; cada registro queda asociado a la transacción que lo originó. Se serializa en dns.edd y puede reconstruirse desde la blockchain.
Matriz Dispersa Ortogonal (tráfico)	Se alimenta exclusivamente de las transacciones auditadas: al iniciar el programa se reconstruye por completo recorriendo los bloques válidos de la blockchain.
BFS y DFS	Se conservan para el análisis de conectividad. El DFS se reutiliza para obtener la única ruta entre dos Data Centers dentro del árbol de expansión mínima.

El sistema conserva los dos perfiles de usuario de las fases anteriores: **Administrador de Data Center** y **Usuario DevOps / Cliente Corporativo**. Antes de detallar las funcionalidades de cada perfil, se describe el flujo integrado que une a las estructuras nuevas con las de la Fase 2:

Flujo Integrado de Operación de un Trabajo

1. Solicitud: un cliente DevOps solicita un trabajo (DEPLOY, MIGRAR, REPLICAR, BACKUP o PARCHE) indicando origen, destino, volumen, prioridad y hostname; el trabajo se inserta en el Min-Heap.

2. Despacho: el administrador despacha el trabajo de mayor prioridad (raíz del Min-Heap); se calcula su ruta por el grafo con BFS, se acumula el volumen en la celda origen–destino de la Matriz Dispersa y, si corresponde, se publica o actualiza el registro en la Tabla Hash del DNS.

3. Transacción: al completarse el trabajo se genera una transacción con todos sus datos y se agrega a la cola de transacciones pendientes; el trabajo queda en estado EJECUTADO.

4. Sellado: cuando el administrador solicita un bloque, se calcula el Árbol de Merkle de las transacciones pendientes, se ejecuta la prueba de trabajo, se agrega el bloque por el frente de la cadena y se guarda en blocks/. Los trabajos incluidos pasan a estado AUDITADO.

5. Optimización: el administrador calcula con Prim o Kruskal la red troncal de menor latencia, útil como árbol de replicación global (los trabajos REPLICAR siguen la troncal) o para identificar los enlaces redundantes.

6. Persistencia: al cerrar el programa, el grafo, la Tabla Hash y el Min-Heap se serializan en memoria y se comprimen con LZW en archivos .edd. Al iniciar, se descomprimen, se valida la blockchain y se reconstruye la Matriz Dispersa de tráfico.

Funciones de Administrador

El administrador del sistema contará con un menú principal dentro de una interfaz gráfica formal e intuitiva que le permitirá gestionar la red global, la seguridad de las operaciones y la persistencia del sistema, además de conservar todas las funcionalidades de la Fase 2 (gestión del backbone, DNS, planificador de trabajos, análisis de conectividad y matriz de tráfico). Las nuevas funcionalidades clave son:

1. Gestión de la Red Backbone (Grafo)

Conserva el registro y eliminación de Data Centers (código, ciudad, país, capacidad en racks) y de enlaces (origen, destino, latencia en ms, ancho de banda en Gbps), incorporando el estado activo o inactivo de cada enlace. La opción Simular Falla de Enlace de la Fase 2 ahora marca el enlace como inactivo, y tanto el BFS/DFS como Prim y Kruskal ignoran los enlaces inactivos, lo que permite estudiar cómo cambia la red troncal ante una falla.

2. Carga Masiva de Datos (JSON)

Procesa el archivo JSON de carga masiva para poblar, en una sola operación, el Grafo (Data Centers y enlaces), la Tabla Hash del DNS y el Min-Heap de trabajos pendientes. El analizador valida la estructura del documento, los tipos de datos y la integridad referencial antes de insertar cada elemento (por ejemplo, un enlace cuyo origen o destino no existe, o un trabajo con un Data Center inexistente, se rechaza). Al finalizar muestra un resumen con la cantidad de elementos cargados y la lista de registros rechazados con su motivo. El historial de tráfico de la Fase 2 deja de cargarse desde un archivo: la Matriz Dispersa se alimenta únicamente de la blockchain. Si existen archivos .edd válidos al iniciar el programa, el estado se restaura desde ellos y la carga masiva es opcional.

3. Red Troncal de Costo Mínimo (Prim y Kruskal)

El administrador selecciona un Data Center de origen y ejecuta el cálculo del Árbol de Expansión Mínima con ambos algoritmos, usando la latencia como peso. Prim se ejecuta desde el origen elegido; Kruskal ordena los enlaces activos por latencia y utiliza una estructura Union-Find implementada manualmente para detectar ciclos. Para cada algoritmo se presenta el orden en que se aceptan los enlaces, los enlaces rechazados por formar ciclo (en Kruskal), la relación padre → hijo de cada Data Center, la latencia total y una comparación de ambos resultados, que deben coincidir en el costo total. Si el grafo no es conexo, Prim informa los Data Centers inalcanzables y Kruskal produce un bosque de expansión mínima, indicando el número de componentes. Además, la opción Ruta en la Troncal calcula con DFS la única ruta entre dos Data Centers dentro del árbol y la compara con la ruta directa del BFS.

4. Despacho de Trabajos con Registro de Auditoría

Mantiene el despacho del Min-Heap de la Fase 2, que ahora genera automáticamente una transacción por cada trabajo ejecutado (datos del trabajo, cliente, hostname, dirección IP, tipo de registro, TTL y fecha de ejecución). El administrador puede consultar la cola de transacciones pendientes y el estado de cada trabajo (EN COLA, EJECUTADO, AUDITADO).

5. Auditoría Blockchain y Árbol de Merkle

Permite al administrador generar un nuevo bloque cada vez que lo solicite, almacenando en el campo DATA todas las transacciones pendientes. El sistema calcula el Árbol de Merkle de dichas transacciones, ejecuta la prueba de trabajo iterando el NONCE hasta obtener un hash con prefijo "0000", inserta el bloque por el frente de la cadena y lo guarda como archivo JSON en la carpeta blocks/. Si no existen transacciones pendientes, no se genera el bloque. Adicionalmente, el administrador puede consultar la cadena, generar el reporte del

Árbol de Merkle del último bloque agregado y ejecutar manualmente la verificación de integridad de toda la cadena.

6. Persistencia y Compresión LZW

Permite guardar manualmente el estado de la red, del DNS y de los trabajos pendientes en los archivos red.edd, dns.edd y trabajos.edd, operación que también se ejecuta de forma automática al cerrar el programa. Para cada archivo, el sistema muestra el tamaño del texto original, el tamaño comprimido y el porcentaje de reducción. Al iniciar el programa, si los archivos existen, se descomprimen y se reconstruyen las estructuras en memoria. Todo el proceso se realiza directamente desde las estructuras en memoria, sin generar archivos JSON o XML intermedios.

7. Módulo de Visualización Graphviz

Genera y exporta dinámicamente la representación visual en tiempo real del Árbol de Expansión Mínima obtenido con Prim, del obtenido con Kruskal, del Árbol de Merkle del último bloque y de la Blockchain completa, mediante la generación automatizada de archivos DOT y su compilación a imágenes PNG/SVG con Graphviz. Los reportes de la Fase 2 continúan disponibles.

Funciones del Módulo Usuario DevOps / Cliente

Los usuarios DevOps y Clientes Corporativos conservan las funcionalidades de la Fase 2 (resolución de nombres DNS, solicitud y seguimiento de trabajos, consulta de rutas y alcance, y consulta de tráfico) y disponen de las siguientes funcionalidades nuevas para consultar y verificar su infraestructura:

1. Iniciar Sesión

Módulo de autenticación que valida las credenciales del usuario (correo electrónico y contraseña) garantizando un acceso restringido y seguro a su perfil corporativo.

2. Consulta de la Red Troncal

Permite consultar la red troncal de menor latencia vigente y obtener la ruta, dentro de ese árbol, entre dos Data Centers (por ejemplo, el origen y el destino de una réplica que desea planificar), con la latencia acumulada de cada salto.

3. Solicitud de Trabajos con Estado de Auditoría

Permite solicitar un trabajo y dar seguimiento a su estado completo: EN COLA, EJECUTADO (transacción pendiente de sellado) y AUDITADO, indicando en este último caso el INDEX del bloque que lo contiene.

4. Historial Auditado de la Infraestructura

Lista, a partir de los bloques válidos de la blockchain, todos los trabajos auditados del cliente (filtrados por su código de cliente) con su fecha de ejecución, Data Centers involucrados y volumen transferido.

5. Verificación de Comprobantes de Auditoría

Dado el identificador de un trabajo, el sistema localiza el bloque que contiene su transacción, muestra su INDEX y su HASH, y comprueba el comprobante recalculando la raíz de Merkle a partir del hash de la transacción y de los hashes hermanos del árbol (prueba de inclusión). El resultado se informa como comprobante válido o no válido, y se advierte si el bloque fue marcado como manipulado.

Especificación de Carga Masiva de Datos (Archivo JSON)

Con el fin de mejorar la legibilidad y de representar de forma natural las relaciones entre entidades, la carga masiva de la Fase 3 se realiza mediante un único archivo JSON, que sustituye a los archivos CSV de la Fase 2 y alimenta tres estructuras al mismo tiempo:

Formato de Archivo: carga_masiva_fase3.json (Alimenta el Grafo, la Tabla Hash y el Min-Heap)

Estructura del documento: un objeto con tres secciones. "backbone", con los arreglos "datacenters" (codigo, ciudad, pais, capacidad_racks) y "enlaces" (origen, destino, latencia_ms, ancho_banda_gbps, activo), donde origen y destino son códigos de Data Centers existentes; "registros_dns" (hostname, direccion_ip, tipo_registro, ttl_segundos, codigo_cliente); y "trabajos" (id_trabajo, tipo, dc_origen, dc_destino, volumen_gb, prioridad, codigo_cliente, hostname). Los enlaces son no dirigidos, por lo que cada uno se inserta en la lista de adyacencia de ambos extremos.

Ejemplo:

```
{
  "backbone": {
    "datacenters": [
      { "codigo": "DC-GUA", "ciudad": "Guatemala", "pais": "Guatemala", "capacidad_racks": 120 },
      { "codigo": "DC-MEX", "ciudad": "Ciudad de México", "pais": "México", "capacidad_racks":
200 },
      { "codigo": "DC-MIA", "ciudad": "Miami", "pais": "EE. UU.", "capacidad_racks": 300 }
    ],
    "enlaces": [
      { "origen": "DC-GUA", "destino": "DC-MEX", "latencia_ms": 18,
        "ancho_banda_gbps": 100, "activo": true },
      { "origen": "DC-GUA", "destino": "DC-MIA", "latencia_ms": 24,
        "ancho_banda_gbps": 400, "activo": true }
    ]
  },
  "registros_dns": [
    { "hostname": "api.cloudgrid.gt", "direccion_ip": "10.0.1.40",
      "tipo_registro": "A", "ttl_segundos": 3600, "codigo_cliente": "CLI001" }
  ],
  "trabajos": [
    { "id_trabajo": 110, "tipo": "MIGRAR", "dc_origen": "DC-GUA", "dc_destino": "DC-MIA",
      "volumen_gb": 120, "prioridad": 2, "codigo_cliente": "CLI001",
      "hostname": "api.cloudgrid.gt" }
  ]
}
```

Reglas de validación: los códigos de Data Center, los hostnames y los identificadores de trabajo deben ser únicos; los enlaces deben tener latencia positiva y referirse a Data Centers existentes (que se cargan primero); el tipo de trabajo debe ser DEPLOY, MIGRAR, REPLICAR, BACKUP o PARCHE. Los registros inválidos no detienen la carga: se omiten y se informan al final.

Especificación de la Blockchain y del Árbol de Merkle

Estructura del Bloque

La blockchain se comporta como una lista enlazada simple en la que cada nodo (bloque) se inserta por el frente; el último bloque generado es la cabeza de la lista y cada bloque conoce el hash de su antecesor. Cada bloque contiene los siguientes campos:

Campo	Descripción
INDEX	Número secuencial del bloque. El bloque génesis tiene índice 0 y cada bloque insertado incrementa este valor en uno.
TIMESTAMP	Fecha y hora de creación del bloque en formato DD-MM-AAAA::HH:MM:SS.
NONCE	Número entero que el sistema itera desde 0, de uno en uno, hasta encontrar un hash que cumpla la prueba de trabajo.
DATA	Arreglo JSON con las transacciones de trabajos ejecutados que estaban pendientes al momento de generar el bloque.
ROOTMERKLE	Hash raíz del Árbol de Merkle construido sobre las transacciones del campo DATA. Se calcula antes de buscar el NONCE.
PREVIOUSHASH	Hash del bloque inmediatamente anterior. En el bloque génesis (INDEX 0) tiene el valor fijo "0000".
HASH	Hash SHA-256 del bloque actual, calculado sobre la concatenación sin espacios de INDEX + TIMESTAMP + NONCE + ROOTMERKLE + PREVIOUSHASH.

Fórmula del hash del bloque: HASH = SHA256(INDEX + TIMESTAMP + NONCE + ROOTMERKLE + PREVIOUSHASH)

Prueba de trabajo: un bloque es válido solo si su HASH comienza con exactamente cuatro ceros ("0000..."). El sistema itera el NONCE desde cero, incrementándolo en uno y recalculando el hash en cada intento, hasta encontrar uno que cumpla la condición, lo que da a la creación de bloques un costo computacional real y vuelve impráctica la alteración de la cadena. El bloque génesis se crea automáticamente la primera vez que se ejecuta el programa, con DATA vacío. Para SHA-256 se permite implementar el algoritmo manualmente o utilizar una librería criptográfica; la blockchain, el Árbol de Merkle, Prim, Kruskal y la compresión LZW deben implementarse manualmente.

Transacciones del Campo DATA

Cada elemento del arreglo DATA representa una transacción de trabajo ejecutado, no la solicitud del trabajo: el trabajo ya existe en el Min-Heap desde que el cliente lo solicita, pero solo se vuelve oficial e inmutable cuando su transacción queda registrada dentro de un bloque. Por ello, cada transacción debe contener toda la información necesaria para que, únicamente a partir de la blockchain, se pueda reconstruir por completo la Matriz Dispersa de tráfico (dc_origen, dc_destino y volumen_gb) y los registros DNS publicados (hostname, direccion_ip, tipo_registro y ttl_segundos), junto con id_trabajo, tipo, codigo_cliente y fecha_ejecucion.

Ejemplo de un bloque almacenado en blocks/bloque_0001.json (el bloque tiene dos transacciones y, por tanto, dos hojas en su Árbol de Merkle):

```
{
  "INDEX": 1,
  "TIMESTAMP": "12-11-2026::10:00:05",
  "NONCE": 70533,
  "DATA": [
    {
      "id_trabajo": 101, "tipo": "MIGRAR", "codigo_cliente": "CLI001",
      "dc_origen": "DC-GUA", "dc_destino": "DC-MIA", "volumen_gb": 120,
      "hostname": "api.cloudgrid.gt", "direccion_ip": "10.0.1.40",
      "tipo_registro": "A", "ttl_segundos": 3600,
      "fecha_ejecucion": "12-11-2026::09:14:22"
    },
    {
      "id_trabajo": 102, "tipo": "DEPLOY", "codigo_cliente": "CLI001",
      "dc_origen": "DC-MEX", "dc_destino": "DC-DFW", "volumen_gb": 40,
      "hostname": "auth-v2.cloudgrid.gt", "direccion_ip": "10.0.2.20",
      "tipo_registro": "A", "ttl_segundos": 3600,
      "fecha_ejecucion": "12-11-2026::09:31:05"
    }
  ],
  "ROOTMERKLE": "87d2d6013a84ea057403d6179b403cd3f24553e92016de48662860d0e1186907",
  "PREVIOUSHASH": "0000a9140f5db02789a93e1b049f8c9439334ec64bcd81e0a0188c693548f364",
  "HASH": "00009c408be14f2c04e19f8d5822d03334e197048fd7ed0905b8a812e551320f"
}
```

Árbol de Merkle por Bloque

Un Árbol de Merkle es un árbol binario en el que cada nodo hoja contiene el hash de un bloque de datos y cada nodo interno contiene el hash combinado de sus dos hijos; el nodo raíz resume criptográficamente todo el conjunto. En el sistema, cada transacción del campo DATA es una hoja del árbol: hoja = SHA256(transacción serializada), donde la serialización es el JSON compacto de la transacción, sin espacios y con los campos en el orden indicado. Cada nodo interno se calcula como SHA256(hash del hijo izquierdo + hash del hijo derecho). Si un nivel tiene una cantidad impar de nodos, el último se duplica para formar el par. La raíz obtenida se almacena en el campo ROOTMERKLE del bloque. El administrador

puede generar el reporte del árbol del último bloque agregado; un bloque con dos transacciones tiene dos hojas.

Protección de Datos y Verificación de Integridad

Los datos de las transacciones no deben poder modificarse directamente en los archivos JSON sin que el sistema lo detecte. Al iniciar el programa con bloques existentes en la carpeta blocks/, el sistema lee todos los archivos en orden de INDEX y verifica cada bloque: (1) la raíz de Merkle recalculada a partir del campo DATA coincide con ROOTMERKLE; (2) el hash recalculado coincide con el campo HASH y comienza con "0000"; y (3) el campo PREVIOUSHASH coincide con el HASH del bloque anterior. Un bloque que no cumple alguna verificación, o que sigue a uno que no la cumple, se considera manipulado.

Inmediatamente después de la verificación se genera automáticamente el reporte de la cadena, en el que los bloques íntegros se colorean de verde y los manipulados de rojo. Únicamente se incorporan al sistema los bloques hasta el último bloque no manipulado, y a partir de las transacciones de esos bloques se reconstruye la Matriz Dispersa de tráfico. Si el archivo dns.edd no existe o está dañado, la Tabla Hash del DNS se reconstruye a partir de los registros contenidos en esos mismos bloques.

Especificación de la Compresión LZW y la Persistencia (.edd)

El estado de la red, del DNS y de los trabajos pendientes se persiste en tres archivos con extensión .edd, cada uno comprimido de forma independiente con el algoritmo LZW (Lempel-Ziv-Welch): red.edd (Data Centers y enlaces del grafo), dns.edd (Tabla Hash) y trabajos.edd (Min-Heap). Estos archivos no son archivos de carga masiva ni auxiliares: representan el estado final de las estructuras al cerrar el programa. Como durante la ejecución el administrador puede agregar, modificar o eliminar Data Centers, enlaces, registros DNS y trabajos, el archivo de carga masiva queda desactualizado, por lo que la compresión se realiza exclusivamente a partir de las estructuras actuales en memoria y nunca se vuelve a consultar el archivo de carga masiva.

Serialización en Memoria

Antes de comprimir, cada estructura se recorre por completo (el grafo recorriendo vértices y listas de adyacencia, la Tabla Hash recorriendo todas sus posiciones y tomando solo las celdas OCUPADAS, y el Min-Heap recorriendo su arreglo) y se transforma en una cadena de texto estructurada que se construye únicamente en memoria. Cada cadena tiene el aspecto de los siguientes documentos y se entrega directamente al algoritmo LZW:

```
{ "red": [ ... ] }
{ "dns": [ ... ] }
{ "trabajos": [ ... ] }
```

No se permite el proceso Estructura en memoria → archivo JSON/XML → LZW → .edd, porque crearía un archivo intermedio innecesario y haría depender la compresión de un archivo externo.

Algoritmo y Formato del Archivo

El compresor inicia con un diccionario de 256 entradas (códigos 0 a 255, un carácter por entrada) y lo amplía con cada nueva secuencia encontrada, asignándole el siguiente código

disponible; el diccionario debe implementarse con una estructura propia (por ejemplo, una lista enlazada, un árbol o la Tabla Hash de la Fase 2), sin contenedores de la STL. El descompresor reconstruye el mismo diccionario a partir de los códigos, por lo que este no se almacena en el archivo. Cada archivo .edd contiene una cabecera con la firma "EDD1", el tamaño del texto original y la cantidad de códigos, seguida de los códigos como enteros sin signo de 16 bits. Si el diccionario alcanza 65 536 entradas, deja de ampliarse. Como ejemplo, la cadena "ABABABA" se comprime en los códigos 65, 66, 256 y 258:

Carácter leído	Secuencia (w + carácter)	Acción	Salida / nueva entrada
A	A	Está en el diccionario	—
B	AB	No está: se emite el código de A	Salida 65; AB = 256
A	BA	No está: se emite el código de B	Salida 66; BA = 257
B	AB	Está en el diccionario	—
A	ABA	No está: se emite el código de AB	Salida 256; ABA = 258
B	AB	Está en el diccionario	—
A	ABA	Está en el diccionario	—
Fin	—	Se emite el código de ABA	Salida 258

Flujo de Compresión y Persistencia

#	Momento	Acción del sistema
1	Primera carga	El sistema lee el archivo de carga masiva JSON y almacena Data Centers, enlaces, registros DNS y trabajos en las estructuras en memoria.
2	Durante la ejecución	El administrador y los usuarios DevOps crean, modifican, eliminan y consultan información. Todas las operaciones se realizan sobre las estructuras en memoria, cuyo estado puede diferir del archivo de carga masiva.
3	Preparación del cierre	Se recorren por completo el grafo, la Tabla Hash y el Min-Heap, y se obtiene la información necesaria para reconstruirlos.
4	Serialización	La información se transforma en una cadena de texto estructurada en memoria; no se genera ningún archivo intermedio.
5	Compresión	Cada cadena se entrega al algoritmo LZW y el resultado se guarda en red.edd, dns.edd y trabajos.edd, sobrescribiendo los anteriores.
6	Siguiente ejecución	Si existen los archivos .edd, se descomprimen con LZW y se reconstruyen las estructuras; luego se verifica la blockchain y se reconstruye la Matriz Dispersa de tráfico.

Área de Reportes

Cada estructura de datos debe contar con un método dedicado capaz de exportar su estado actual a un archivo en formato DOT y compilarlo a una imagen PNG/SVG mediante llamadas al comando nativo 'dot -Tpng <archivo>.dot -o <imagen>.png'. La interfaz gráfica debe renderizar estos diagramas directamente en pantalla, y los reportes deben actualizarse automáticamente cada vez que cambie la estructura correspondiente:

Reporte 1: Árbol de Expansión Mínima con Prim

Debe generarse la vista del grafo del backbone con la latencia en milisegundos como etiqueta de cada enlace. Las aristas que Prim incorpora al árbol se resaltan en verde, con mayor grosor, y se numeran según el orden de selección; los enlaces descartados se muestran punteados. El reporte indica el Data Center de origen y la latencia total.

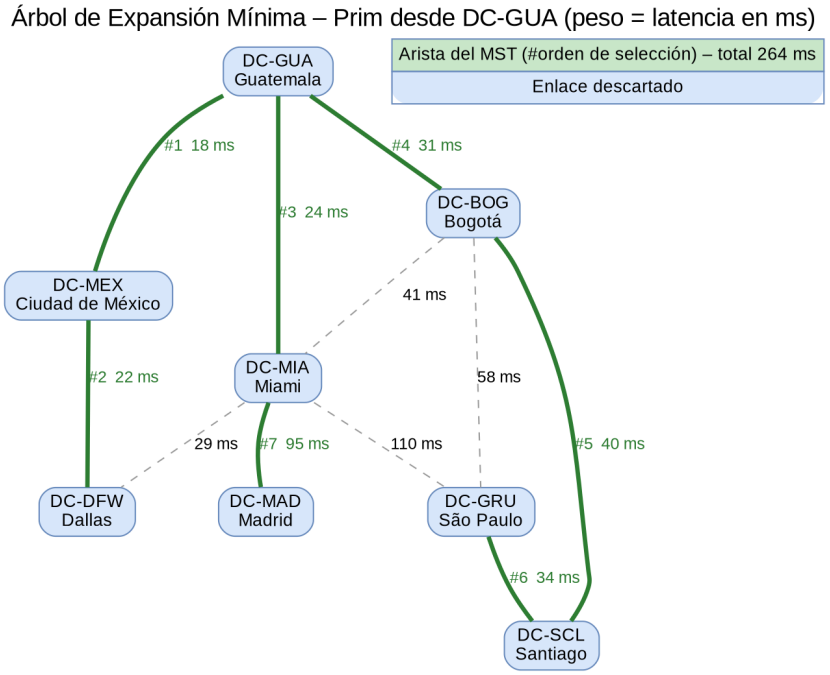


Figura 1: Árbol de Expansión Mínima obtenido con Prim desde DC-GUA (total 264 ms).

Reporte 2: Árbol de Expansión Mínima con Kruskal

Debe generarse la misma vista del grafo, numerando cada enlace según su posición en el orden creciente de latencia. Los enlaces aceptados se marcan en verde y los rechazados por formar un ciclo, en rojo y punteados, de modo que se aprecie el trabajo de la estructura Union-Find. Ambos reportes deben mostrar el mismo costo total.

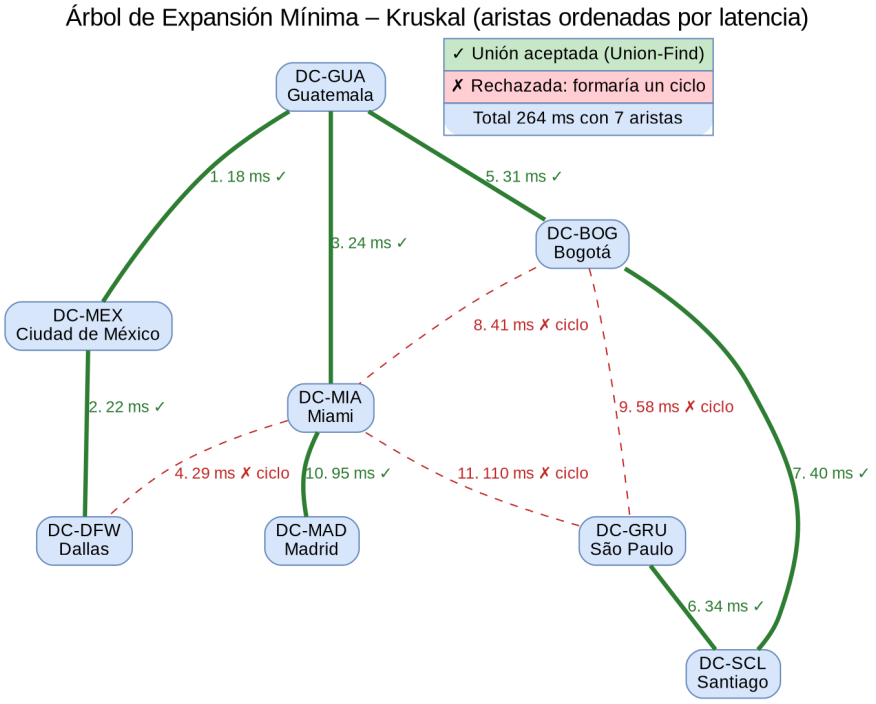


Figura 2: Árbol de Expansión Mínima obtenido con Kruskal; enlaces aceptados y rechazados.

Reporte 3: Árbol de Merkle del Último Bloque

Debe mostrarse el árbol binario del último bloque agregado: en las hojas, el hash de cada transacción identificada por el id del trabajo; en los nodos internos, el hash combinado de sus hijos; y en la raíz, el valor almacenado en ROOTMERKLE. Los hashes pueden abreviarse en el reporte, pero el cálculo se realiza siempre con el valor completo.

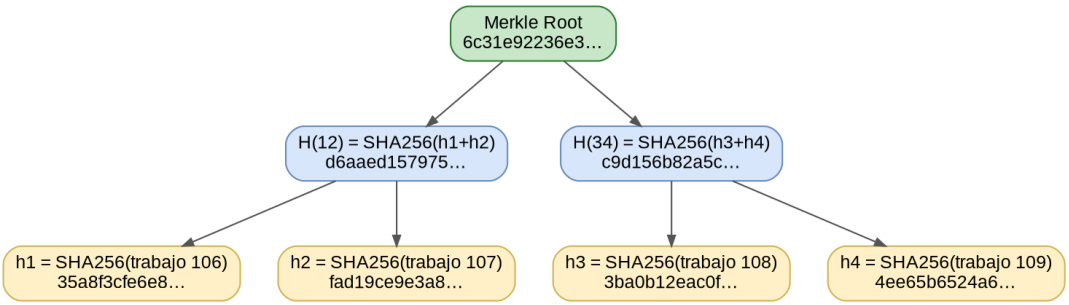


Figura 3: Árbol de Merkle de un bloque con cuatro transacciones.

Reporte 4: Blockchain (Bloques Íntegros en Verde y Manipulados en Rojo)

Debe mostrarse la cadena completa como lista enlazada, desde la cabeza (último bloque) hasta el bloque génesis, indicando en cada bloque su INDEX, TIMESTAMP, NONCE, ROOTMERKLE, PREVIOUSHASH y HASH. Los bloques cuya verificación es correcta se colorean de verde y los manipulados (o posteriores a uno manipulado) de rojo. Este reporte se genera automáticamente al iniciar el programa y cada vez que se agrega un bloque.

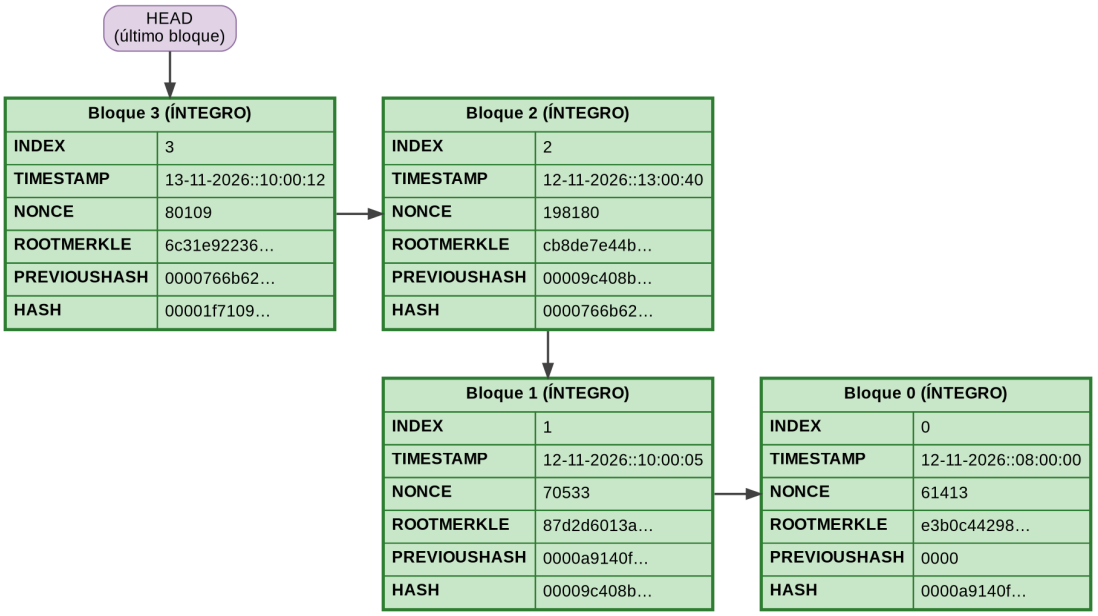


Figura 4: Blockchain íntegra; todos los bloques superan la verificación.

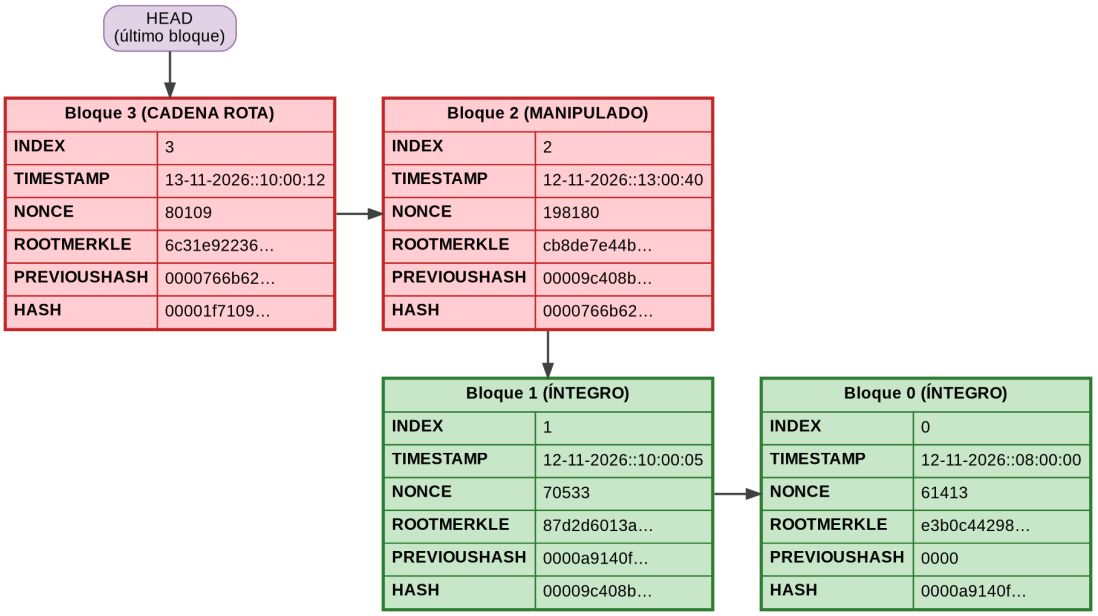


Figura 5: Blockchain tras modificar manualmente el campo DATA del bloque 2 en su archivo JSON.

4.2 Alcance del proyecto

Las estructuras de datos deben ser desarrolladas completamente por los estudiantes sin utilizar librerías o módulos predefinidos de C++ que implementen directamente estas estructuras (std::map, std::unordered_map, std::vector, std::list, std::queue, std::stack, std::set, std::priority_queue, entre otros). Toda la lógica de enlazado, inserción, eliminación, recorrido, cálculo de los árboles de expansión mínima, construcción del árbol de Merkle, encadenamiento de bloques y compresión debe ser manual. Para el cálculo de SHA-256 se

permite el uso de una librería criptográfica, pero no para las estructuras ni algoritmos mencionados.

Los reportes gráficos son esenciales para la evaluación. Si no se presenta el reporte de alguna estructura, se anularán los puntos correspondientes. El código debe ser original, individual y debidamente comentado.

4.2 Requerimiento técnicos

Realizar la aplicación en C++

El sistema se desarrollará completamente en C++, implementando una aplicación con interfaz gráfica que gestione todas las funcionalidades descritas. Se utilizarán las capacidades nativas de C++ para el manejo de archivos JSON, archivos binarios (.edd) y procesamiento de texto. El código deberá ser modular, con funciones claramente definidas para cada operación del sistema y con una separación clara entre las estructuras de datos y la interfaz.

Realizar los reportes en Graphviz

Todos los reportes gráficos deberán generarse utilizando exclusivamente Graphviz. El sistema C++ creará archivos DOT con la descripción de los grafos y ejecutará los comandos necesarios para generar las imágenes en formato PNG o PDF. Los reportes se actualizarán automáticamente cada vez que se modifiquen las estructuras de datos, permitiendo una visualización en tiempo real del estado del sistema.

5. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

5.1 Proyectos Individuales

Los proyectos individuales permiten a los estudiantes desarrollar autonomía y responsabilidad sobre su propio trabajo. En este tipo de proyectos, cada estudiante debe gestionar su tiempo, investigar, planificar y ejecutar todas las fases del proyecto de manera independiente.

5.1.1 Autogestión del Tiempo

Los estudiantes deben crear un cronograma personal para cumplir con los plazos establecidos. Esto les ayuda a mejorar su disciplina y capacidad de priorización, habilidades esenciales en cualquier entorno profesional.

5.1.2 Responsabilidad y Compromiso

En un proyecto individual, el estudiante asume la totalidad de las responsabilidades, desde la investigación hasta la entrega final. Esto fomenta el sentido de compromiso y permite una mayor personalización en la solución del problema planteado.

5.1.3 Resolución de Problemas

Trabajar de manera independiente impulsa a los estudiantes a buscar soluciones de forma creativa y a enfrentarse a los desafíos sin depender del apoyo constante de otros. Esto fortalece su capacidad para resolver problemas de manera autónoma.

5.1.4 Reflexión Personal

Al concluir el proyecto, el estudiante realiza una autoevaluación, reflexionando sobre sus decisiones, lo aprendido y las áreas en las que podría mejorar. Esta práctica promueve el desarrollo continuo y el autoaprendizaje.

5.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Implementación de estructuras de datos		
Árbol de Expansión Mínima – Algoritmo de Prim	10	
Árbol de Expansión Mínima – Algoritmo de Kruskal (con Union-Find)	10	
Blockchain	12	
Árbol de Merkle	5	
Compresión LZW	7	
Sub-Total	**44**	
2. Funcionalidades del sistema		
Funciones del Módulo Administrador (Red troncal, Auditoría, Persistencia)	18	
Funciones del Módulo Usuario DevOps (Troncal, Estado de auditoría, Comprobantes)	12	
Sub-Total	**30**	
3. Reportes gráficos con Graphviz		
Reporte de Árbol de Expansión Mínima (Prim)	4	
Reporte de Árbol de Expansión Mínima (Kruskal)	4	
Reporte de Árbol de Merkle	2	
Reporte de Blockchain	6	
Sub-Total	**16**	
4. Manual técnico y documentación	**10**	
TOTAL	**100**	

5.3 Observaciones

- El lenguaje de programación para esta fase será C++.
- La herramienta de desarrollo de reportes gráficos será exclusivamente Graphviz.
- La entrega se realizará por medio de GitHub, en el mismo repositorio de las fases anteriores, cuyo nombre debe ser EDD_2S2026_PY_#carnet (ejemplo: EDD_2S2026_PY_202102771). Dentro de él se creará una carpeta con el nombre EDD_Fase3.
- Por medio de UEDI se hará entrega del link del repositorio.
- Deberá agregar a su respectivo auxiliar a su repositorio como colaborador:
    - user → Sección B
    - user → Sección C
    - user → Sección A
- Los repositorios deben mantenerse en privado para evitar copias de código entre estudiantes.
- La carpeta blocks/ debe encontrarse en la raíz del proyecto (dentro de EDD_Fase3) y contener, como mínimo, el bloque génesis y los bloques generados durante la demostración.
- Se debe realizar el manual técnico en formato Markdown dentro de una carpeta llamada documentacion, dentro de la carpeta de la respectiva fase, explicando la implementación de cada estructura, los algoritmos de Prim, Kruskal y LZW, su complejidad y las decisiones de diseño tomadas.
- Todas las dudas durante el proceso de desarrollo deberán realizarse por medio de los foros de UEDI de la respectiva sección.
- Las copias detectadas serán penalizadas con calificación de 0 puntos y reportadas a la Escuela de Ciencias y Sistemas.
- Se debe realizar commits frecuentes en GitHub para demostrar el avance progresivo del proyecto.
- El código debe estar debidamente comentado y seguir buenas prácticas de programación.
- Se debe incluir el archivo de prueba carga_masiva_fase3.json con datos de ejemplo para demostrar la funcionalidad de carga masiva.

FECHA DE ENTREGA: xx de xx del 2026

No.	Criterio de evaluación	Puntaje máximo asignado	Excelente 100 % – 85 %	Satisfactorio 84 % – 61 %	Necesita mejorar 60 % – 0 %	Punteo obtenido
1	Implementación de estructuras de datos y algoritmos	44				
1.1	Árbol de Expansión Mínima – Algoritmo de Prim	10	Implementa correctamente el algoritmo de Prim sobre la lista de adyacencia del backbone, utilizando la latencia como peso y considerando únicamente los enlaces activos. Permite seleccionar el Data Center de origen, registra el orden de selección de los enlaces, obtiene las relaciones padre → hijo, calcula correctamente la latencia total e identifica los Data Centers inalcanzables. Utiliza estructuras auxiliares propias y controla correctamente la memoria dinámica. Rango: [10.00 – 8.50 pts]	Implementa los componentes principales de Prim y obtiene el árbol de expansión mínima en los casos generales. El costo es mayormente correcto, pero presenta errores menores en el orden de selección, manejo de enlaces inactivos, relaciones padre → hijo, grafos desconectados, presentación de resultados o administración de memoria. Rango: [8.40 – 6.10 pts]	El algoritmo está incompleto o no funciona. Genera árboles o costos incorrectos, incluye enlaces inactivos, no permite seleccionar el origen, no identifica Data Centers inalcanzables, utiliza contenedores prohibidos de la STL o presenta fallos graves de memoria. Rango: [6.00 – 0 pts]	
1.2	Árbol de Expansión Mínima – Algoritmo de Kruskal y Union-Find	10	Implementa correctamente Kruskal, ordenando los enlaces activos por latencia y utilizando una estructura Union-Find desarrollada manualmente. Detecta ciclos, diferencia los enlaces aceptados de los rechazados, registra el orden de evaluación, calcula el costo total y genera un bosque de expansión mínima cuando el grafo	Kruskal funciona en los casos principales y utiliza una estructura propia para administrar conjuntos disjuntos. Presenta errores menores en el ordenamiento de enlaces, detección de ciclos, manejo de empates, conteo de componentes, comparación con Prim o administración de	El algoritmo está incompleto, acepta enlaces que forman ciclos, obtiene un costo incorrecto, no implementa Union-Find manualmente, no controla grafos desconectados o utiliza estructuras prohibidas de	

			no es conexo. El costo coincide con Prim cuando el grafo es conexo. Rango: [10.00 – 8.50 pts]	memoria. Rango: [8.40 – 6.10 pts]	la STL. Rango: [6.00 – 0 pts]	
1.3	Blockchain con prueba de trabajo	12	Implementa la blockchain como una lista enlazada simple con inserción por el frente. Cada bloque contiene correctamente INDEX, TIMESTAMP, NONCE, DATA, ROOTMERKLE, PREVIOUSHASH y HASH. Ejecuta la prueba de trabajo hasta obtener un hash SHA-256 con prefijo `0000`, crea correctamente el bloque génesis, persiste cada bloque en un archivo JSON y verifica la integridad de la cadena al iniciar. Detecta bloques manipulados y excluye de la reconstrucción los bloques inválidos y sus sucesores. Rango: [12.00 – 10.20 pts]	La cadena permite crear, enlazar, almacenar y consultar bloques. La prueba de trabajo funciona en los casos principales, pero presenta errores menores en la persistencia, validación de campos, creación del bloque génesis, orden de lectura, detección de alteraciones o propagación del estado manipulado. Rango: [10.08 – 7.32 pts]	La blockchain está incompleta o inoperativa. Los bloques no se enlazan correctamente, el hash no sigue la fórmula indicada, no cumple la prueba de trabajo, no persiste los archivos JSON, no crea el bloque génesis o no detecta manipulaciones. Rango: [7.20 – 0 pts]	
1.4	Árbol de Merkle y prueba de inclusión	5	Construye correctamente el Árbol de Merkle a partir de las transacciones serializadas. Calcula las hojas con SHA-256, combina los hashes hijos en el orden correcto, duplica el último nodo cuando la cantidad es impar y obtiene una raíz que coincide con ROOTMERKLE. Implementa correctamente la prueba de inclusión mediante hashes hermanos y detecta cambios en las transacciones. Rango: [5.00 – 4.25 pts]	Construye el árbol y calcula la raíz en los casos principales, pero presenta errores menores en la serialización, el manejo de cantidades impares, el orden de concatenación, la identificación de hojas, la prueba de inclusión o la integración con la blockchain. Rango: [4.20 – 3.05 pts]	El árbol está incompleto, calcula raíces incorrectas, no procesa todas las transacciones, no maneja cantidades impares, no implementa la prueba de inclusión o no integra ROOTMERKLE con los bloques. Rango: [3.00 – 0 pts]	
1.5	Compresión, descompresión	7	Implementa manualmente la compresión y descompresión LZW	La compresión y descompresión funcionan con las estructuras	LZW está incompleto, pierde información, no	

	LZW y persistencia en archivos `.edd`		con un diccionario dinámico de 256 entradas iniciales y límite de 65 536 códigos. Genera correctamente la cabecera `EDD1`, almacena el tamaño original, la cantidad de códigos y los códigos de 16 bits. Serializa directamente desde memoria y restaura correctamente `red.edd`, `dns.edd` y `trabajos.edd`. Muestra el tamaño original, tamaño comprimido y porcentaje de reducción. Rango: [7.00 – 5.95 pts]	principales, pero existen errores menores en la cabecera, reconstrucción del diccionario, tratamiento de archivos dañados, estadísticas de compresión, serialización o restauración de alguna estructura. Rango: [5.88 – 4.27 pts]	reconstruye las estructuras, utiliza archivos JSON o XML intermedios, no genera el formato `.edd` requerido o utiliza una estructura de diccionario prohibida. Rango: [4.20 – 0 pts]	
	Sub-Total de puntos del área 1	44				
2	Funcionalidades del sistema	30				
2.1	Funciones del módulo Administrador	18	El módulo integra correctamente la gestión de Data Centers, enlaces activos e inactivos, DNS, Min-Heap, Matriz Dispersa, carga masiva JSON, despacho de trabajos, generación de transacciones, estados de auditoría, creación y verificación de bloques, Árbol de Merkle, Prim, Kruskal, persistencia LZW y generación de reportes. La carga valida tipos de datos, duplicados e integridad referencial, y presenta un resumen de registros aceptados y rechazados. La interfaz es clara, estable y coherente. Rango: [18.00 – 15.30 pts]	Implementa las funciones principales del administrador y permite completar el flujo general de operación. Presenta errores menores de integración, validación, interfaz, gestión de estados, carga JSON, recuperación de archivos, manejo de errores o actualización automática de reportes. Rango: [15.12 – 10.98 pts]	Faltan varias funciones esenciales, el flujo integrado no puede completarse, la carga JSON no valida los datos, los trabajos no generan transacciones, la auditoría no funciona, la persistencia falla o la interfaz presenta errores que impiden operar el sistema. Rango: [10.80 – 0 pts]	
2.2	Funciones del módulo Usuario	12	Implementa autenticación y acceso restringido por cliente. Permite	Las funciones principales del usuario están disponibles, pero	El módulo está incompleto, no restringe la	

	DevOps / Cliente Corporativo		solicitar trabajos, consultar los estados EN COLA, EJECUTADO y AUDITADO, consultar la red troncal, obtener rutas entre Data Centers, visualizar el historial auditado desde bloques válidos y verificar comprobantes mediante pruebas de inclusión de Merkle. Presenta correctamente el INDEX y HASH del bloque relacionado y advierte cuando el bloque fue manipulado. Rango: [12.00 – 10.20 pts]	presentan errores menores de autenticación, filtrado por cliente, solicitud y seguimiento de trabajos, consulta de rutas, historial auditado, visualización del bloque o verificación del comprobante. Rango: [10.08 – 7.32 pts]	información por cliente, no permite solicitar o dar seguimiento a trabajos, no consulta la troncal, no obtiene el historial desde la blockchain o no verifica los comprobantes de auditoría. Rango: [7.20 – 0 pts]	
	Sub-Total de puntos del área 2	30				
3	Reportes gráficos con Graphviz	16				
3.1	Reporte del Árbol de Expansión Mínima con Prim	4	Genera automáticamente el archivo DOT y la imagen PNG/SVG del backbone. Muestra las latencias de los enlaces, resalta en verde y con mayor grosor los enlaces seleccionados por Prim, numera el orden de selección, representa los enlaces descartados con estilo punteado e incluye el Data Center de origen y la latencia total. Se visualiza desde la interfaz y se actualiza al cambiar la estructura. Rango: [4.00 – 3.40 pts]	El reporte representa el árbol correctamente en términos generales, pero presenta errores menores en colores, estilos, numeración, etiquetas, costo total, Data Center de origen, visualización o actualización automática. Rango: [3.36 – 2.44 pts]	El reporte no existe, no compila, no representa el resultado real de Prim, muestra información escrita manualmente o carece de los elementos necesarios para identificar el árbol obtenido. Rango: [2.40 – 0 pts]	
3.2	Reporte del Árbol de Expansión Mínima con Kruskal	4	Genera automáticamente el archivo DOT y la imagen PNG/SVG. Numera los enlaces según el orden creciente de latencia, diferencia en verde los enlaces aceptados y en rojo	El reporte muestra el resultado principal de Kruskal, pero presenta errores menores en el orden de los enlaces, colores, estilos, enlaces rechazados, costo total, componentes,	El reporte no existe, no compila, no coincide con la ejecución de Kruskal, utiliza datos escritos manualmente o no permite distinguir los	

			punteado los rechazados por formar ciclos, muestra el costo total y representa correctamente el bosque cuando el grafo está desconectado. Rango: [4.00 – 3.40 pts]	visualización o actualización automática. Rango: [3.36 – 2.44 pts]	enlaces aceptados de los rechazados. Rango: [2.40 – 0 pts]	
3.3	Reporte del Árbol de Merkle del último bloque	2	Representa correctamente el Árbol de Merkle del último bloque. Identifica las hojas mediante el id del trabajo, muestra los hashes de las transacciones, los nodos internos y ROOTMERKLE, conservando la estructura real utilizada para el cálculo. Se genera y actualiza automáticamente desde la interfaz. Rango: [2.00 – 1.70 pts]	El árbol representa las transacciones principales, pero presenta errores menores en etiquetas, niveles, hashes abreviados, duplicación del último nodo, identificación de transacciones o actualización automática. Rango: [1.68 – 1.22 pts]	El reporte no existe, no compila, no corresponde al último bloque, no representa la estructura utilizada para calcular ROOTMERKLE o utiliza información escrita manualmente. Rango: [1.20 – 0 pts]	
3.4	Reporte de la Blockchain	6	Representa la cadena completa desde la cabeza hasta el bloque génesis. Cada bloque muestra INDEX, TIMESTAMP, NONCE, ROOTMERKLE, PREVIOUSHASH y HASH. Los bloques íntegros aparecen en verde y los manipulados, junto con sus sucesores inválidos, aparecen en rojo. El reporte se genera al iniciar el programa y cada vez que se agrega un bloque. Rango: [6.00 – 5.10 pts]	El reporte representa la cadena y sus enlaces, pero presenta errores menores en los campos mostrados, orden de los bloques, colores, propagación del estado manipulado, visualización o actualización automática. Rango: [5.04 – 3.66 pts]	El reporte no existe, no compila, no representa toda la cadena, no diferencia bloques íntegros y manipulados, no coincide con los archivos almacenados en `blocks/` o utiliza datos escritos manualmente. Rango: [3.60 – 0 pts]	
	Sub-Total de puntos del área 3	16				
4	Manual técnico, organización, repositorio y documentación	10				

4.1	Manual técnico, repositorio de GitHub y sustentación	10	Presenta un manual técnico profesional en formato Markdown dentro de `documentacion/`. Explica la arquitectura, estructuras propias, memoria dinámica, Prim, Kruskal, Union-Find, blockchain, Árbol de Merkle, LZW, serialización, archivos `.edd`, carga JSON, Graphviz, complejidades y decisiones de diseño. El repositorio privado está correctamente organizado, contiene `EDD_Fase3`, `blocks/`, `carga_masiva_fase3.json` y commits frecuentes. El código es modular, original, comentado y la sustentación demuestra dominio técnico completo. Rango: [10.00 – 8.50 pts]	La documentación explica los componentes esenciales y el repositorio contiene la mayoría de los entregables, pero presenta errores menores de organización, profundidad, análisis de complejidad, decisiones de diseño, frecuencia de commits, comentarios, evidencias o claridad durante la sustentación. Rango: [8.40 – 6.10 pts]	La documentación es incompleta, confusa o inexistente; no explica las estructuras y algoritmos utilizados; el repositorio está desorganizado; faltan archivos requeridos; existen pocos commits; el código carece de modularidad o el estudiante no puede explicar el funcionamiento de su solución. Rango: [6.00 – 0 pts]	
	Sub-Total de puntos del área 4	10				
	TOTAL	100				